

National Undergraduate Programme in Mathematical Sciences

Introduction to Programming

End-semester Examination, Semester I, 2025–2026

Date : November 24, 2025

Time : 0930–1230

Marks : 100

Weightage : 40%

-
- This paper has two parts.
 - There are 5 questions in Part A, each worth 6 marks. For this part, provide the solution and if necessary, a short explanation justifying your answer.
 - Part B is worth 70 marks. For this part, provide crisp and short programs. Syntax should more or less correspond to Haskell, but minor inaccuracies like wrong indentation will be ignored. You can assume standard library functions, but if you use something not taught in class, you must explain briefly what it does.
-

Part A

1. Give any type that can be assigned to the function f defined to be

```
f x = fst x ++ [not (snd x)]
```

2. Give the index of the pair $(5, 3)$ in the following list, remembering that list indices start with 0.

```
[(x,y) | y <- [0..], x <- [y..(2*y)]]
```

3. Consider the following (non-binary) tree data structure.

```
data Tree = Node Int [Tree]
```

Give five values belonging to this type.

4. Define a function `height :: Tree → Int` for the above data type.
5. What is the behaviour of the following piece of code.

```
replicateM 10 (do
  x <- readLn
  print (map toUpper x)
)
```

Part B

1. Write a program of type `IO ()` with the following behaviour. It reads a line of input which is a number n . Then it reads n lines of input, each containing a number. It finally prints a list which consists of the numbers read, in reverse order, and stops. So if the first line of input is 8, and the subsequent numbers input are 0, 1, 1, 2, 3, 5, 8 and 13, then the program prints `[13,8,5,3,2,1,1,0]`.
(20 marks)

2. Consider the `myTail`, `myInit` functions defined as follows.

```
myTail, myInit :: [a] → [a]
```

```
myTail []      = []      myInit []      = []
myTail (x:xs)  = xs      myInit [x]     = []
                                myInit (x:y:ys) = x:myInit (y:ys)
```

Recall the built-in function `foldr`.

```
foldr :: (a → b → b) → b → [a] → b
foldr f v []      = v
foldr f v (x:xs)  = f x (foldr f v xs)
```

Define `myTail` and `myInit` using `foldr`.

Hint: You may have to come up with an appropriate pair v and a function f that produces pairs, and define your desired functions as `fst . foldr f v`.
(20 marks)

3. For the `Tree` data type defined earlier, we can define a *pre-order traversal* as follows.

```
preorder :: Tree → [Int]
preorder (Node x ts) = x: concatMap preorder ts

preorder (Node 2 [ Node 3 [Node 4 []],
                    Node 5 [Node 6 [], Node 7 [Node 8 [], Node 9 []]]
                ]) = [2,3,4,5,6,7,8,9]
```

Does the above function run in linear time (in terms of the number of nodes in the tree)? Justify your answer.

If it is not linear, define a version that runs in linear time. You may have to define the following function (with the indicated behaviour) without using `preorder`.

```
preordersInto :: [Tree] → [Int] → [Int]
preordersInto ts l = concatMap preorder_ ts l
```

(30 marks)